Docker & MLOps Study Guide: From Code to Container

1. Intuitive Metaphor: The Standardized Shipping Container

Imagine you bake a complex cake recipe in your home kitchen. If you send only the written recipe to a friend, their cake might fail because their oven runs hotter, their mixing bowl is smaller, or they use a different brand of flour.

In traditional software, sending raw code (`app.py`, `models.py`) to a cloud server or a client often fails due to matching operating system mismatches, missing system libraries, or conflicting Python package versions ("it works on my machine" syndrome).

Docker packages the recipe *and* the entire kitchen—the operating system (Debian Linux), system binaries (`gcc`, `curl`), exact Python version (3.11), installed packages (`requirements.txt`), and your exact project code (`.`)—into a single, immutable box called a **Container Image**.

2. Anatomy of What Just Happened in Docker

Plaintext

None

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      DOCKER BUILD & RUN LIFECYCLE                        │
├──────────────────────┬───────────────────────────────────────────────────┤
│ Dockerfile Step      │ Physical Execution Action                         │
├──────────────────────┼───────────────────────────────────────────────────┤
│ FROM python:3.11     │ Downloads a minimal Linux OS pre-configured with Python  │
│ WORKDIR /app         │ Creates `/app` inside the virtual Linux filesystem       │
│ RUN apt-get...       │ Installs C++ compilation tools (`build-essential`)       │
│ COPY req.txt .       │ Copies package bounds into the isolated container        │
│ RUN pip install      │ Installs exact versions of DuckDB, XGBoost, Pytest, etc. │
│ COPY . .             │ Copies engine source code, `src/`, `evals/`, `config/`   │
```

```
| EXPOSE 8501      | Opens port 8501 for Streamlit networking traffic
|
| CMD ["streamlit"]| Configures the default execution command when
booted     |
└──────────────────┴──────────────────────────────────────────────
────────┘
```

When you ran `docker build -t cagequant-engine .`:

1. Docker read your `Dockerfile` line by line inside an isolated build engine (BuildKit).
2.
3. It assembled a multi-layered Linux image filesystem containing every dependency your engine needs to run.
4.

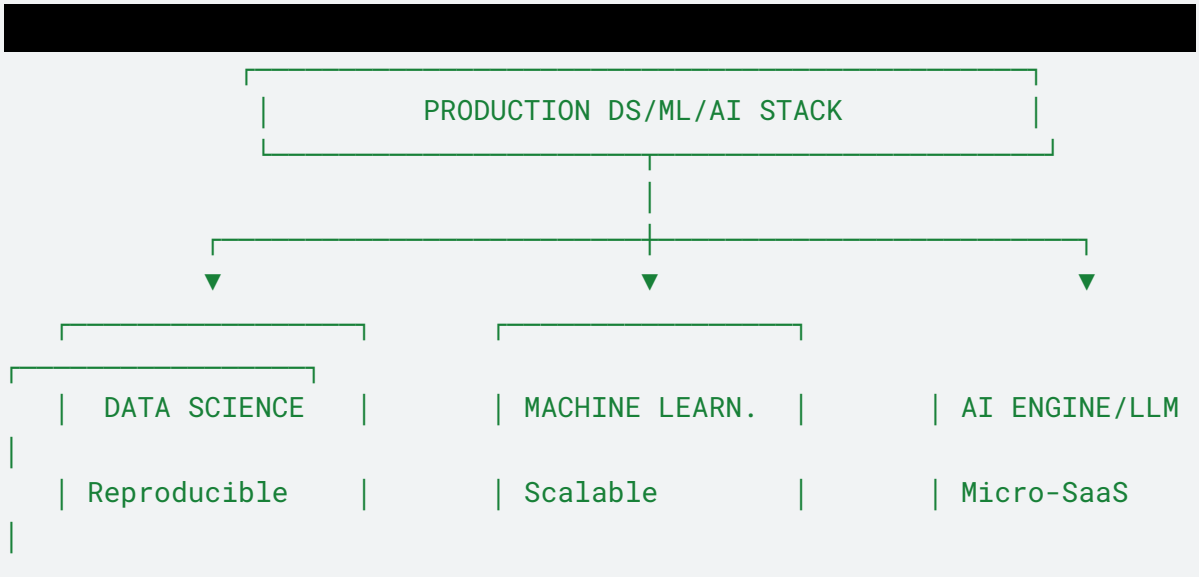
When you run `docker run -p 8501:8501 cagequant-engine`:

1. Docker boots that container image in less than two seconds.
2.
3. It maps port `8501` inside the Linux container directly to port `8501` on your macOS host machine, allowing you to access the app seamlessly at `http://localhost:8501`.
4.

3. How Docker Connects to Data Science, Machine Learning & AI Engineering

Plaintext

None



```
   | DuckDB queries  |       | inference API   |       | tool
endpoints  |
   | across hardware |       | inside containers|      | & FastMCP
apps  |
   └─────────────────┘       └─────────────────┘
└─────────────────┘
```



- **Data Science (DS):** Guarantees that DuckDB analytical queries, random seeds, and statistical calculations produce 100% reproducible results regardless of whether the execution machine is a Mac ARM chip, an Intel Windows server, or an AWS Linux instance.
-
- **Machine Learning (ML):** ML models require strict C/C++ libraries (e.g., OpenMP for XGBoost, BLAS/LAPACK for NumPy/SciPy). Docker isolates these low-level dependencies so model inference code never breaks during deployment.
-
- **AI & LLM Engineering (AI):** Modern AI microservices, RAG systems, and FastMCP tool agents run inside containerized environments (on Railway, Render, AWS ECS, or Hugging Face Spaces) to serve real-time API requests safely without exposing server host resources.
-

4. Lifecycle Placement: Why We Did Docker in Stage 4.4 (Not Stage 1)

In software engineering and MLOps, containerizing prematurely before securing a fully passing local codebase leads to "build thrashing"—spending hours rebuilding multi-gigabyte Docker images every time a minor bug or syntax error occurs.

Following the **Pocock AI Coding Playbook**:

1. **Stages 4.1–4.3 (Tracer Slice First):** Built the complete quantitative engine, FastMCP agent tools, receipt auditor, and golden evals locally, achieving **21 passing tests**.
2.
3. **Stage 4.4 (MLOps Hardening):** Containerized the app *only after* proving the code interface was 100% green.
4.

Phase 4 Achievement & Phase 5 Action Plan

Complete Phase 4 Recap

- **Architecture Decision Records:** Locked ADRs 001 through 014 in `config/decision-log.md`.
-

- **System Boundaries:** Updated PRD.md, design-doc.md, config/ubiquitous_language.md, and LEGAL.md.
-
- **Agentic Analytics Package:** Built src/agent/ featuring FastMCP tool interfaces (tools.py), an analytical copilot (copilot.py), and a structured numerical receipt auditor (auditor.py).
-
- **Deterministic Evals:** Built evals/golden_dataset.json and evals/test_agent_evals.py to benchmark agent performance.
-
- **TDD & MLOps Infrastructure:** Configured root conftest.py, passed **21 Pytest tests**, wrote requirements.txt, created Dockerfile, and configured .github/workflows/ci.yml.
-

Phase 5 Execution: Portfolio Showcase & Omnichannel Launch

Phase 5 focuses on transforming this engineered repository into a high-converting technical portfolio piece and Medium case study.

Plaintext

None

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                          PHASE 5 DELIVERABLES ROADMAP                                │
├──────────────────────┬──────────────────────────────────────────────────────────────┤
│ Deliverable 1        │ GitHub Repository `README.md` (Technical Landing Page)       │
├──────────────────────┼──────────────────────────────────────────────────────────────┤
│ Deliverable 2        │ Medium Technical Case Study Article                          │
├──────────────────────┼──────────────────────────────────────────────────────────────┤
│ Deliverable 3        │ LinkedIn Executive Summary & Portfolio Post                  │
└──────────────────────┴──────────────────────────────────────────────────────────────┘
```

[redacted]

1. High-Impact GitHub README.md Blueprint

Your repository README.md serves as the primary technical landing page for enterprise hiring managers, recruiters, and prospective clients:

- **Header & Status Badges:** Build status (CI: Passing), Python version (3.11), Pytest coverage (21/21 Green), and Docker build state.
- **Executive Summary:** Framing CageQuant as an enterprise Decision Intelligence Engine for 10-second combat telemetry forecasting, stochastic risk modeling, and verified Agentic Analytics.
- **System Architecture Matrix:** Text-based block diagrams illustrating data flow from DuckDB → XGBoost → Monte Carlo → FastMCP Tools → Receipt Auditor → Streamlit UI.
- **Model Calibration Story:** Emphasizing model evaluation via **Brier Skill Score** ($BSS > 0.0$) against empirical baseline win rates, avoiding naive 0.25 coin-flip targets.
- **Agentic Anti-Hallucination Moat:** Demonstrating how the receipt auditor validates 100% of generated numerical claims against ground-truth tool outputs within a $\pm 1.5\%$ tolerance.
- **Reproducibility & Execution Instructions:** Clear steps for running tests locally via pytest and running the containerized app via docker run.

2. Medium Technical Case Study Narrative

Structure your Medium article around these core technical pillars:

- **Title Concept:** *Building an Enterprise Decision Intelligence Engine: Calibrated XGBoost, Monte Carlo Risk Simulation, and Anti-Hallucination Agentic Analytics*
- **Section Breakdown:**

1. **The Business & Tactical Problem:** Translating continuous, chaotic combat dynamics into structured 10-second interval telemetry.
2. **Data & Governance Layer:** DuckDB in-memory analytical queries governed by semantic contracts (config/metrics.yml).
3. **Calibrated ML & Risk Simulation:** Predicting interval win probabilities via XGBoost, validating calibration via Brier Skill Score ($BSS > 0.0$), and simulating $N = 1,000$ Monte Carlo bout iterations with Gaussian damage volatility.

7. **The Agentic AI Moat:** Shifting from passive LLM summarization to active tool execution (FastMCP) and structured receipt-style numerical verification (`auditor.py`).
8.
9. **Cost-Aware MLOps & Evals:** Running deterministic CI tests with mocked LLMs on GitHub Actions alongside live evaluation datasets (`evals/`).
10.

3. Future Micro-SaaS Template Extension Ideas

Now that the CageQuant pattern is complete, this exact codebase structure serves as your reusable template for future commercial micro-SaaS applications:

Plaintext

None

```
┌──────────────────────────────────────────────────────────────────────────
──────────┐
│                  REUSABLE MICRO-SAAS ARCHITECTURE TEMPLATE
│
├──────────────────────┬──────────────────────────────────────────────────
──────────┤
| Domain Component   | CageQuant Engine ➔ Future Micro-SaaS Product
|
├──────────────────────┼──────────────────────────────────────────────────
──────────┤
| Data Layer         | DuckDB Telemetry ➔ Supabase / PostgreSQL
Database       |
| Scoring Engine     | XGBoost Win Prob ➔ Customer Churn / Pricing
Elasticity  |
| Risk Engine        | Monte Carlo Bout ➔ Financial Portfolio Risk /
LTV Sim    |
| Agentic AI Layer   | FastMCP Fight Brief ➔ Automated B2B Executive
Briefs    |
| Monetization       | Open Portfolio ➔ Lemon Squeezy MoR Webhooks +
Auth       |
└──────────────────────┴──────────────────────────────────────────────────
──────────┘
```

How would you like to proceed: shall we draft the production `README.md` for your CageQuant repository first, or write the complete outline and draft for your **Medium Technical Case Study**?

The Docker Intuition: What Actually Changed?

To understand what Docker did when your browser showed the exact same Streamlit dashboard at `http://localhost:8501`, consider this analogy:

Think of your code like a **Microsoft Word document** created on your Mac using custom fonts you manually installed.

- **Running without Docker** is like emailing the raw `.docx` file to a friend on Linux. If their machine lacks those exact fonts or has a different Word version, the layout breaks or refuses to open ("works on my machine").
- **Running with Docker** is like exporting that document into a **standalone, self-contained PDF**. When anyone opens that PDF on Linux, Windows, or AWS, it looks and functions identically because every font, rendering rule, and asset is baked inside.

When you launched `docker run -p 8501:8501 cagequant-engine`:

1. Your browser connected to an isolated Linux operating system running inside Docker memory.
2. That virtual environment had its own isolated Python 3.11 runtime, C++ compilers, DuckDB instance, and XGBoost binaries completely separated from your Mac's global environment.
3. It executed `streamlit run app.py` inside Linux and forwarded only the visual UI stream to your Mac's browser port (`8501`).

The UI looks identical to you because **the engine inside the container executed perfectly**. You can now deploy this exact Docker image to AWS ECS, GCP Cloud Run, or Railway, and it will execute with zero OS-level dependency bugs.